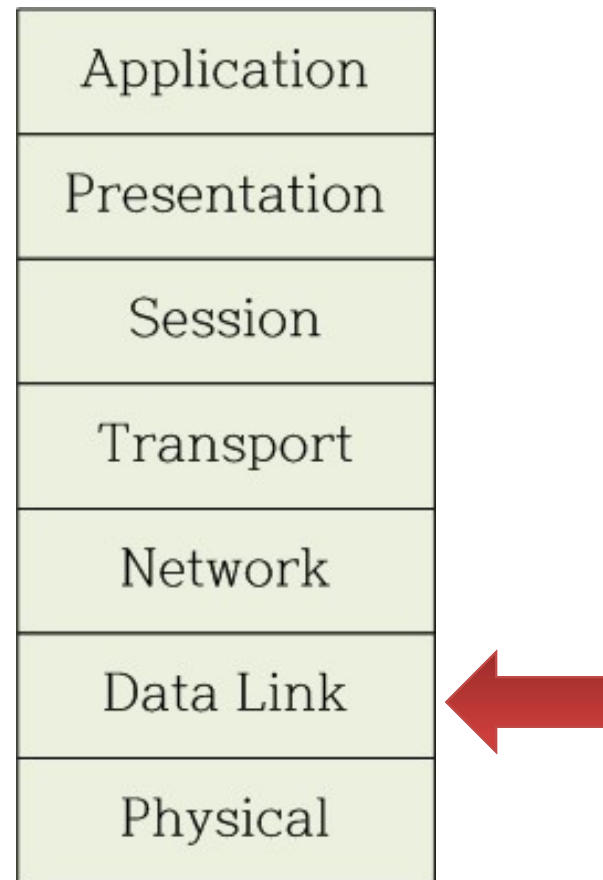


Framing / Error Control

We are here

- OSI 7 layer model



Framing

Why Framing?

- Bits are transmitted continuously → bit stream
 - 010110110001100100100100100111101001011
- Without framing, the receiver cannot tell where one message ends and another begins
 - e.g. Sender sends message A, then message B
 - Receiver sees one continuous bit stream → cannot separate A from B
- Error detection/correction also needs a defined unit to work on
- The first thing is to do is to divide the bit stream into meaningful block of data → "frame"

Reading by Punctuating

“Elizabeth, New Jersey, when my mother was being raised there in a flat over her father’s grocery store, was an industrial port a quarter the size of Newark, dominated by the Irish working class and their politicians and the tightly knit parish life that revolved around the town’s many churches, and though I never heard her complain of having been pointedly ill-treated in Elizabeth as a girl, it was not until she married and moved to Newark’s new Jewish neighborhood that she discovered the confidence that led her to become first a PTA “grade mother,” then a PTA vice president in charge of establishing a Kindergarten Mothers’ Club, and finally the PTA president, who, after attending a conference in Trenton on infantile paralysis, proposed an annual March of Dimes dance on January 30 – President Roosevelt’s birthday – that was accepted by most schools.”

- Philip Roth, “A plot against America”

Framing

- Fixed-size framing
- Variable-size framing

Fixed-size Framing

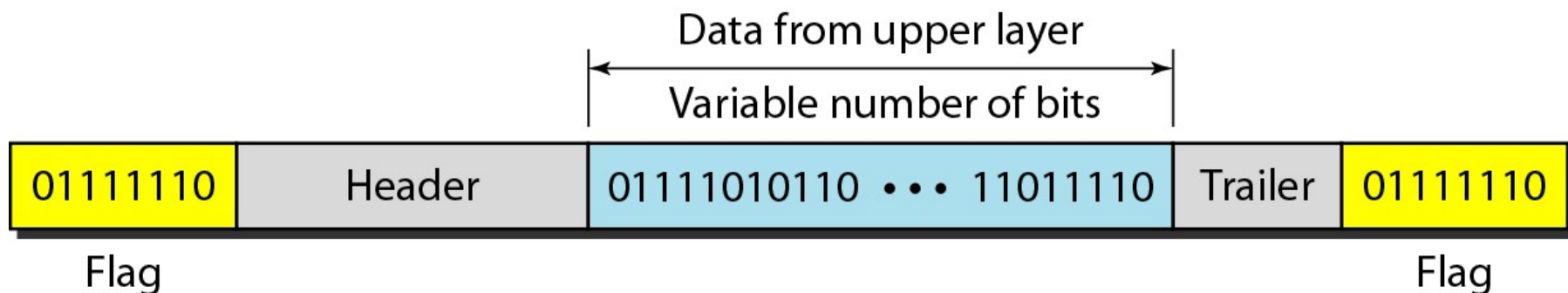
- Fixed number of bits
- No delimiter necessary
- May cause inefficiency
 - Suppose the frame size is 1000 bits
 - If I have only 50 bits to send, the frame still has to be 1000 bits → padding, bandwidth waste
 - If I have 5000 bits to send, the data must be split into multiple frames → fragmentation

Variable-size Framing

- Each frame can have different number of bits
- Delimiter necessary to identify frames
- Can build frames according to the data sent → efficient
 - If I have 50 bits to send → I can make the frame 50 bits

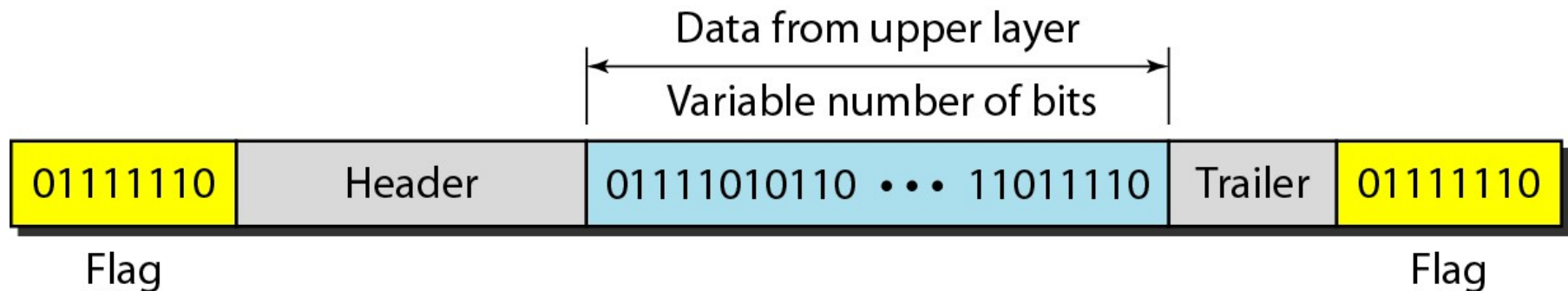
Variable-size Framing

- Flag: marker that points the beginning and the end of a frame
- Header: bits inserted before the data for control purposes
- Trailer: bits inserted after the data for control purposes
- Data/Payload: data to be sent



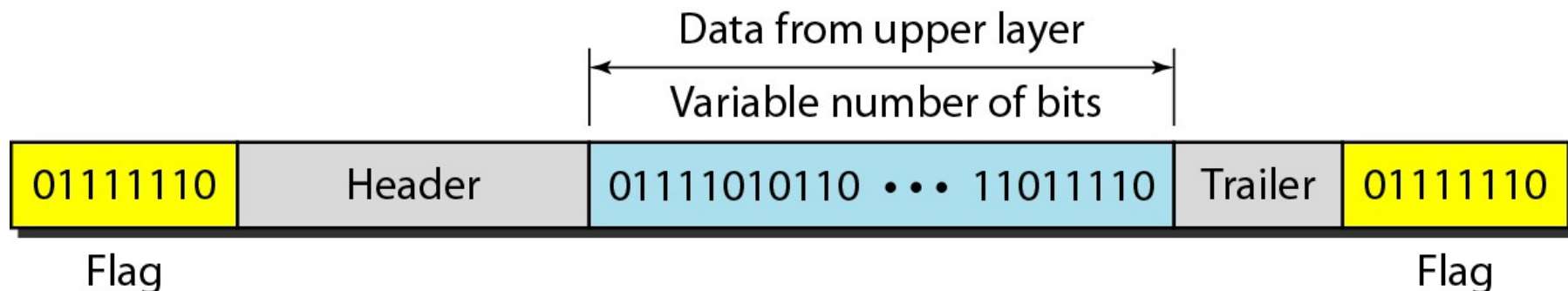
Frame Structure

- Defines size and meanings of fields in a frame



Frame Structure

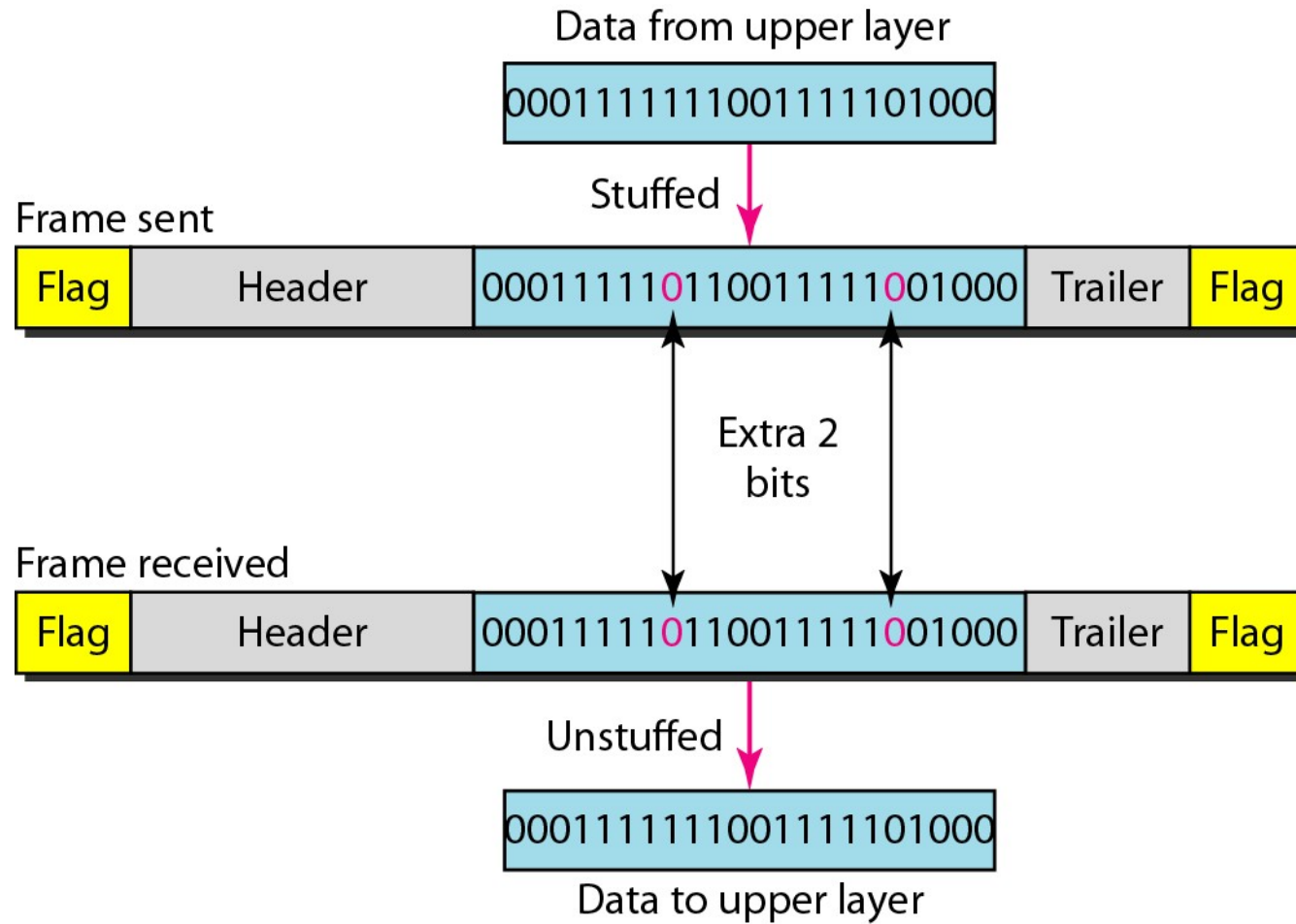
- Start/end marker
 - e.g. 01111110
- What if "01111110" appears in the payload?
 - Suppose payload is "0010101111110100"
 - Assume there is no header/trailer
 - Then the frame is "01111110001010111111010001111110"
 - What is the problem?



Bit Stuffing

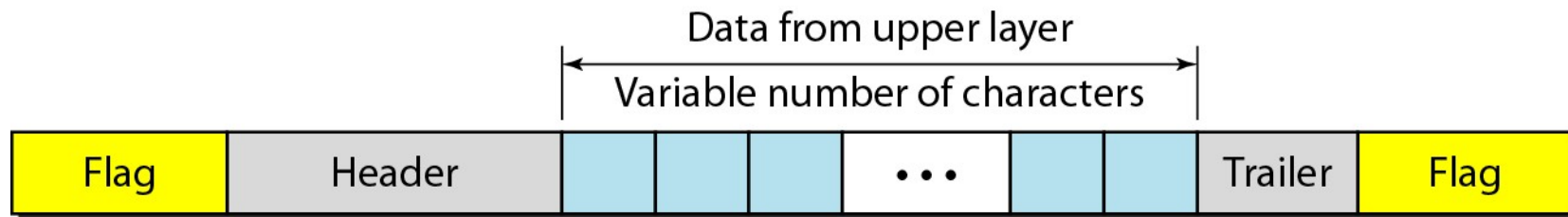
- Problem
 - If the marker pattern appears in the payload, the receiver will think it as an end marker
 - leads to wrong framing
- Solution
 - Ensure the marker pattern does not appear in the payload
 - If “11111” is found, add a “0” after the pattern
 - Bit stuffing
 - The receiver removes “0” after “11111”, since it is the stuffed bit

Bit Stuffing

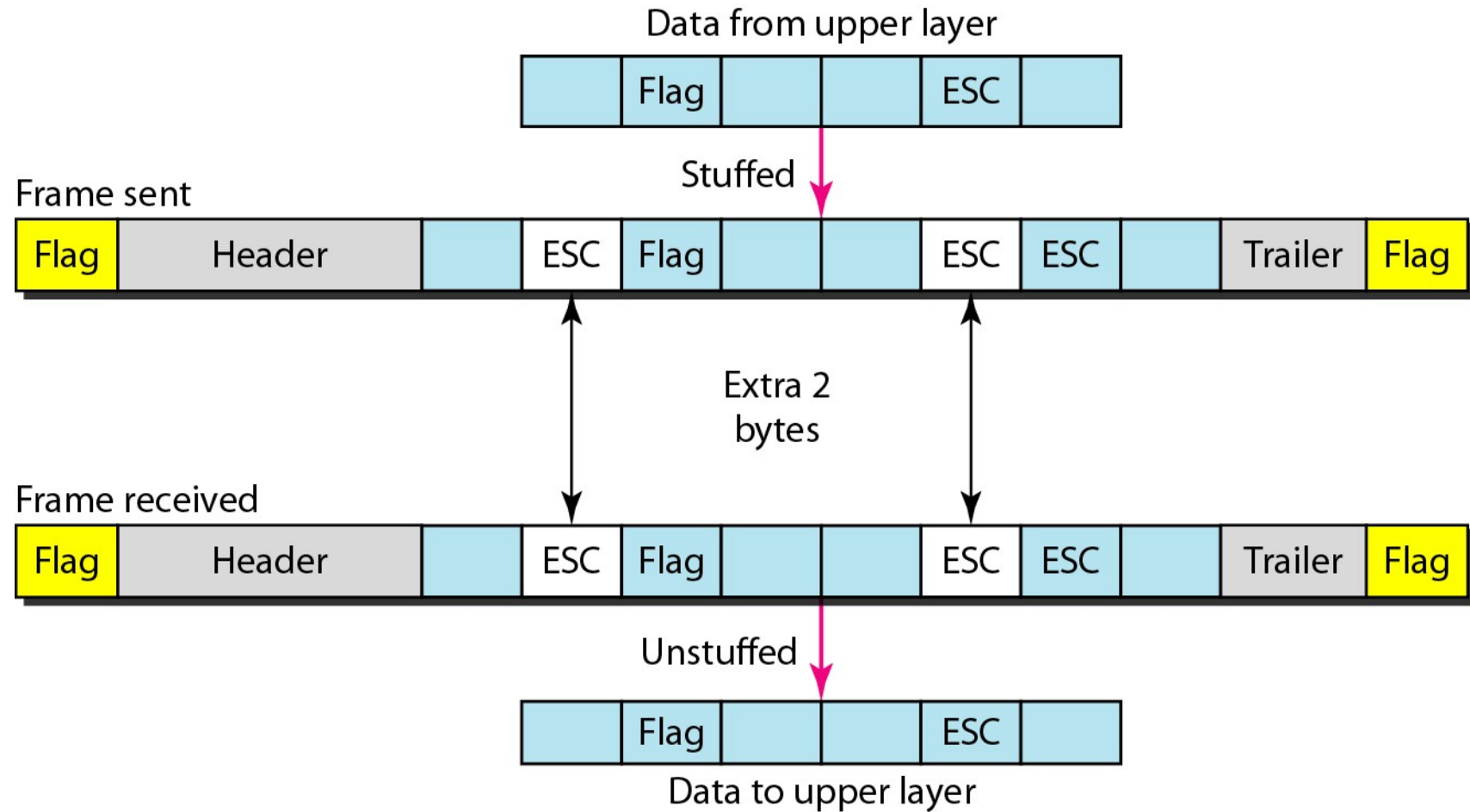


Byte Stuffing

- The same purpose as bit stuffing
- Insert a byte after a marker byte pattern
- Used in systems that process data in unit of bytes
 - e.g. PPP, (Point-to-Point Protocol), SLIP (Serial Line Internet Protocol)



Byte Stuffing



Byte Stuffing

- PPP (Point-to-Point) Protocol is used in dial-up Internet, DSL, serial links
- PPP uses 7E as start/end marker
- If 7E appears in the payload, it is changed to 7D 5E
- If 7D appears in the payload, it is changed to 7D 5D
- 7E → 7D 5E
- 7D → 7D 5D



Error Control

Error Control

- Error detected → What should the receiver do?
- For reliable communication,
 - The receiver should tell the sender to **repeat** frame transmission
- Automatic Repeat reQuest (ARQ)
 - Note: ARQ is a general concept used across multiple layers (Transport layer also uses ARQ-like error-control)

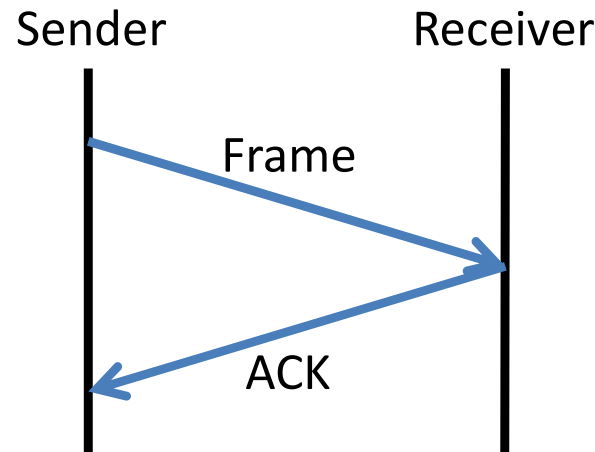
ARQ

- ARQ Types
 - Stop-and-Wait
 - Go-back-N
 - Selective Repeat

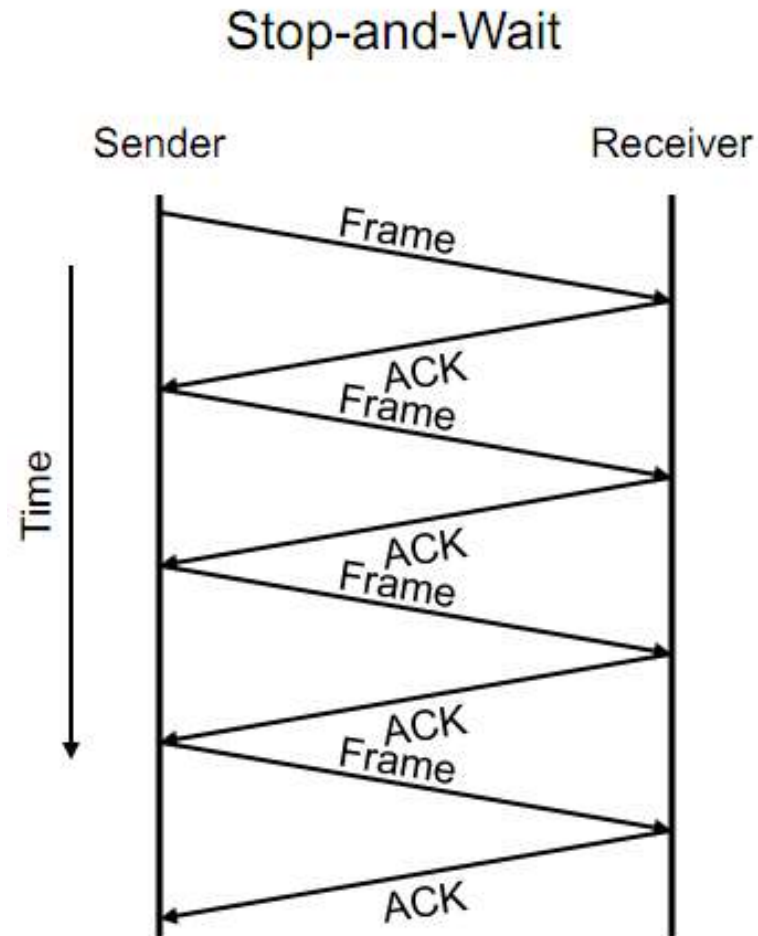
Stop-and-Wait

ACK

- Acknowledgement (ACK) frame
 - Receiver sends an ACK frame to the sender if a frame is successfully received
 - When sender receives the ACK frame, it knows that the sent frame is successfully received at the receiver

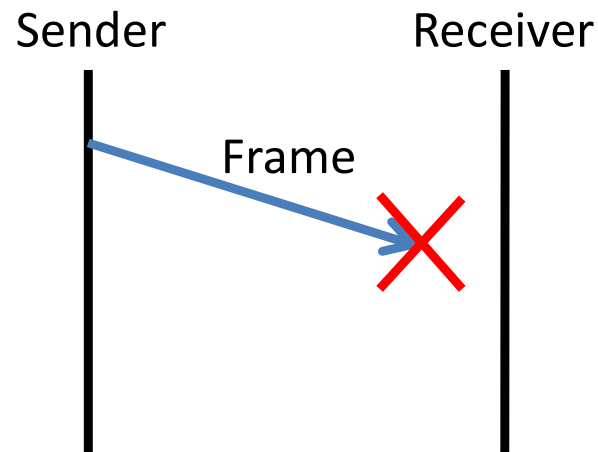


Normal case



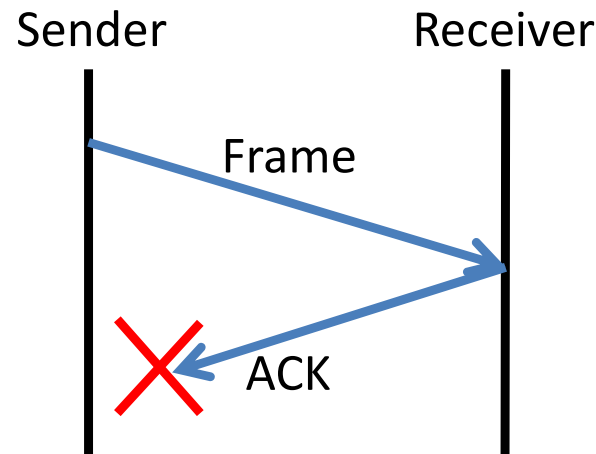
Error case

- What happens if a frame has an error?
 - The receiver does not accept the frame
 - ACK is not sent



Error case

- It is also possible that the frame is correctly received, but the ACK is lost



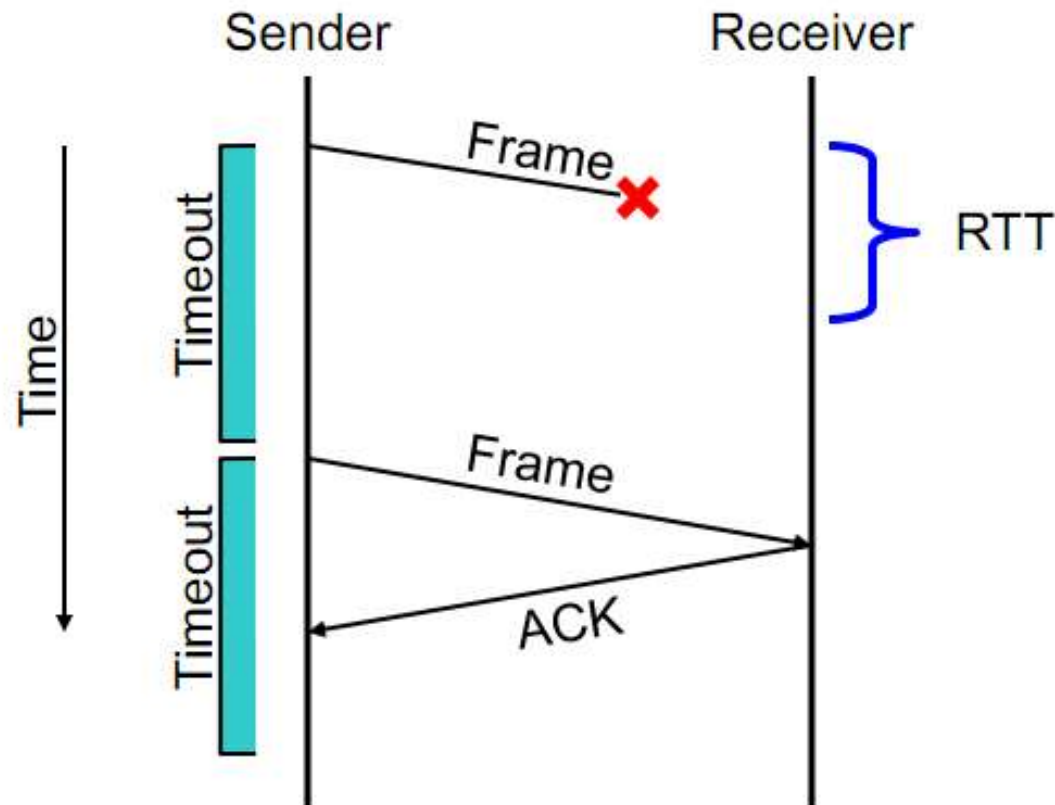
- To the sender, the two cases look the same

Sender's action

- After sending a frame, wait for T seconds
 - T is called the "timeout period"
- If ACK comes in this duration, send the next frame
- If ACK does not arrive in this duration ("timeout"), the sender thinks the frame is lost
→ retransmit the frame

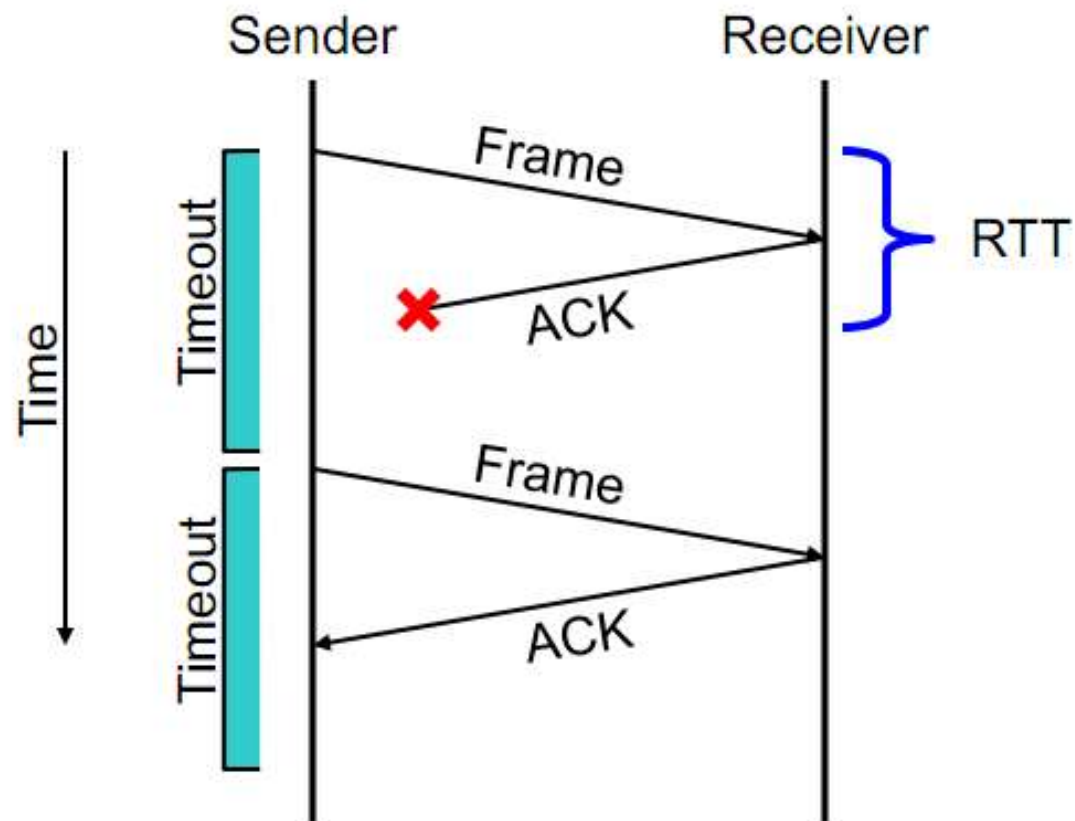
Retransmission cases

- Case 1: frame is lost



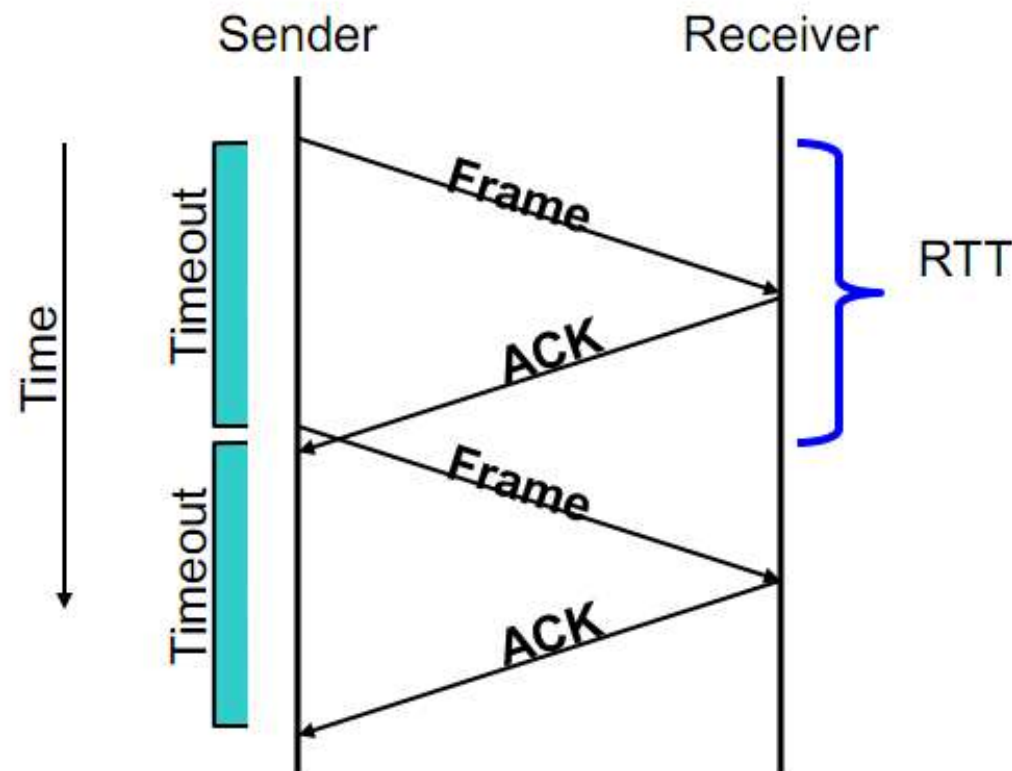
Retransmission cases

- Case 2: frame not lost, but ACK is lost



Retransmission cases

- Case 3: frame not lost, ACK not lost, but ACK arrived later than the timeout
 - Problem: timeout period is too short!



Timeout

- What is the proper timeout period?
- Timeout should be based on RTT (Round-Trip Time)
- RTT: Duration of time between the sender starts sending a frame and finishes receiving an ACK
 - Transmission time of frame +
 - Propagation time of frame +
 - Transmission time of ACK +
 - Propagation time of ACK +
 - Processing time at sender and receiver
- Typical timeout period: $T = \text{RTT} + \text{margin}$
 - margin is to account for unpredicted delays (e.g. processing time)

Receiver's perspective

- A frame is arrived
- Later, another frame is arrived
 - Is this the next frame?
 - Or is this the same frame that has been retransmitted?

Cases

- Normal case
 - S: sends frame
 - R: receives frame, sends ACK
 - S: receives ACK sends next frame
 - R: receives frame
-
- In this case, the frame is a new frame

Cases

- Retransmission case 1: frame is lost
- S: sends frame
- R: (does not receive frame)
- S: timeout called, retransmit frame
- R: receives frame, sends ACK
- S: receives ACK, sends next frame
- In this case, the frame is a new frame

Cases

- Retransmission case 2: ACK is lost
 - S: sends frame
 - R: receives frame, sends ACK
 - S: does not receive ACK, timeout, retransmit frame
 - R: receives frame, sends ACK
-
- In this case, the frame is a retransmitted frame

Cases

- Retransmission case 3: ACK is arrived after timeout
- S: sends frame
- R: receives frame, sends ACK
- S: timeout called, retransmit frame
- S: receives ACK, sends next frame
- R: receives frame, sends ACK
- R: receives frame, sends ACK
- In this case, the frame is a retransmitted frame

Receiver's perspective: Summary

- A frame arrives, so sends an ACK
- Later, another frame arrives
- The receiver does not know if it is a new frame or a retransmitted frame!

Solution: Sequence Number

- Include sequence number in the frame header
- Sequence number is incremented in each frame
- The range of sequence number: $[0...N]$
 - The next of frame(N) is frame(0)
- The ACK also has a sequence number
 - It indicates the sequence number of "next expected frame"
 - If receiver receives frame(0), it sends ACK(1)
 - ACK(1) means "I received frame(0), now send me frame(1)"
- Note: ACK convention varies by protocol
 - Here: ACK(n) = "next expected frame is n"
 - Other protocols may define ACK(n) as "frame(n) received"

Solution: Sequence Number

- Sequence number for stop-and-wait ARQ: [0,1]
- $\text{frame}(0) \rightarrow \text{frame}(1) \rightarrow \text{frame}(0) \rightarrow \text{frame}(1) \rightarrow \dots$

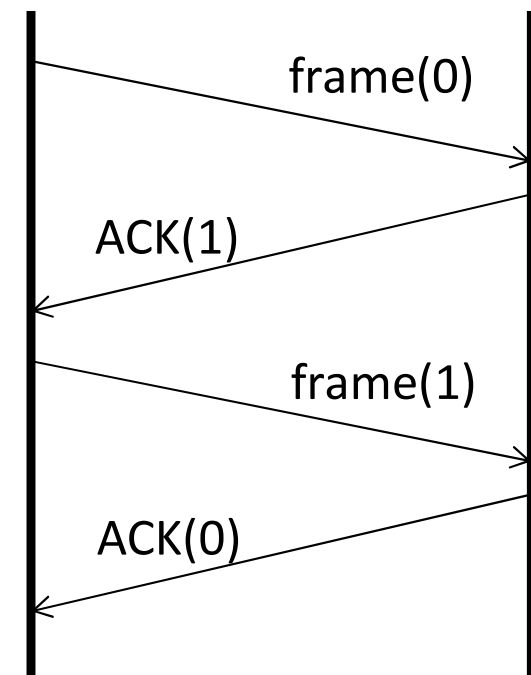
Stop-and-Wait: Frame structure

- Frame(0)



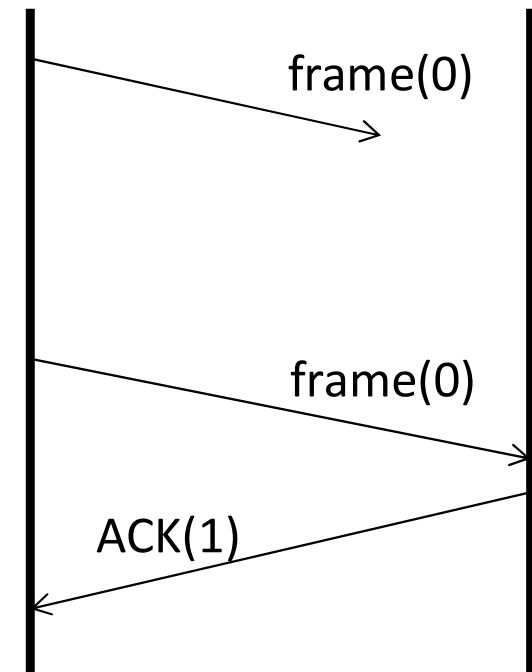
- Transmission sequence

- S: sends frame(0)
- R: sends ACK(1)
 - means the sender should send frame(1)
- S: sends frame(1)
- R: sends ACK(0)
 - means the sender should send frame(0)



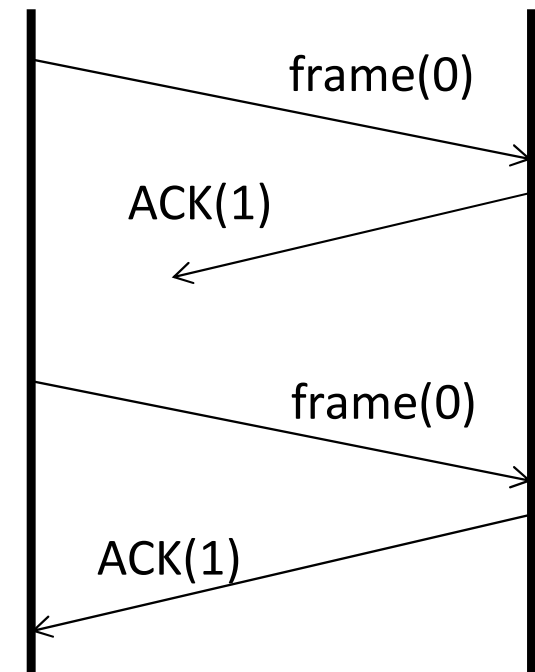
Stop-and-Wait: Error case

- Retransmission case 1: frame lost
 - S: sends frame(0)
 - R: does not receive
 - S: timeout called, retransmits frame(0)
 - R: receives frame(0), sends ACK(1)
 - S: receives ACK(1)
- No problem



Stop-and-Wait: Error case

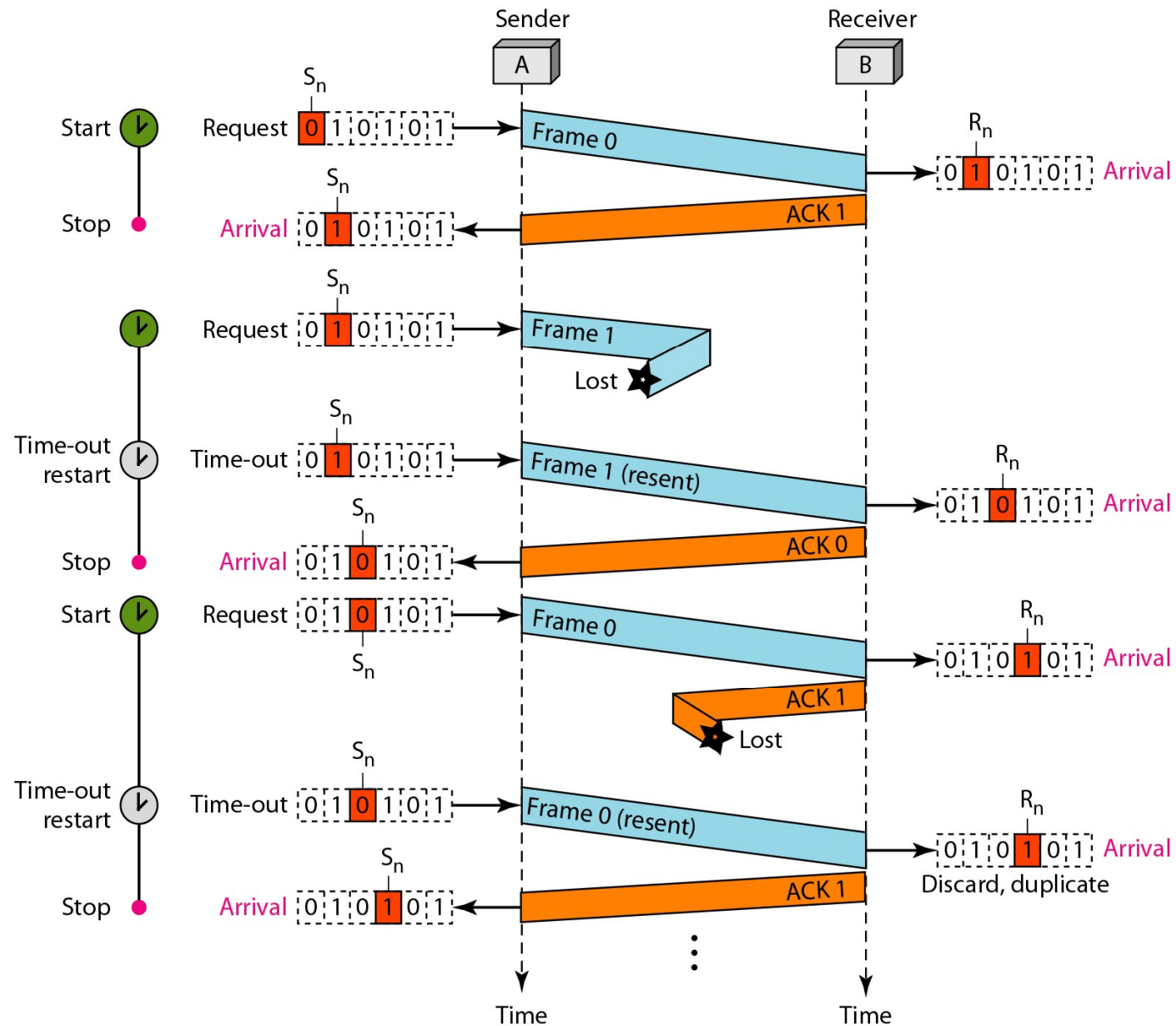
- Retransmission case 2: ACK lost
 - S: sends frame(0)
 - R: receives frame(0), sends ACK(1) → lost
 - S: timeout called, retransmits frame(0)
 - R: receives frame(0), sends ACK(1)
 - S: receives ACK(1)
- Problem solved



Stop-and-Wait: Error case

- Retransmission case 3: ACK arrives after timeout
 - S: sends frame(0)
 - R: receives frame(0), sends ACK(1)
 - S: timeout called, retransmits frame(0)
 - S: receives ACK(1), sends frame(1)
 - R: receives frame(0), sends ACK(1), discards frame(0)
 - Receiver already has frame(0) and is waiting for frame(1)
 - R: receives frame(1), sends ACK(0)
 - S: receives ACK(1), discards ACK(1)
 - Sender already received ACK(1)
 - S: receives ACK(0), sends frame(0)
 - frame(0) is a new frame that comes after frame(1)
- Problem solved

Stop-and-Wait: Error case

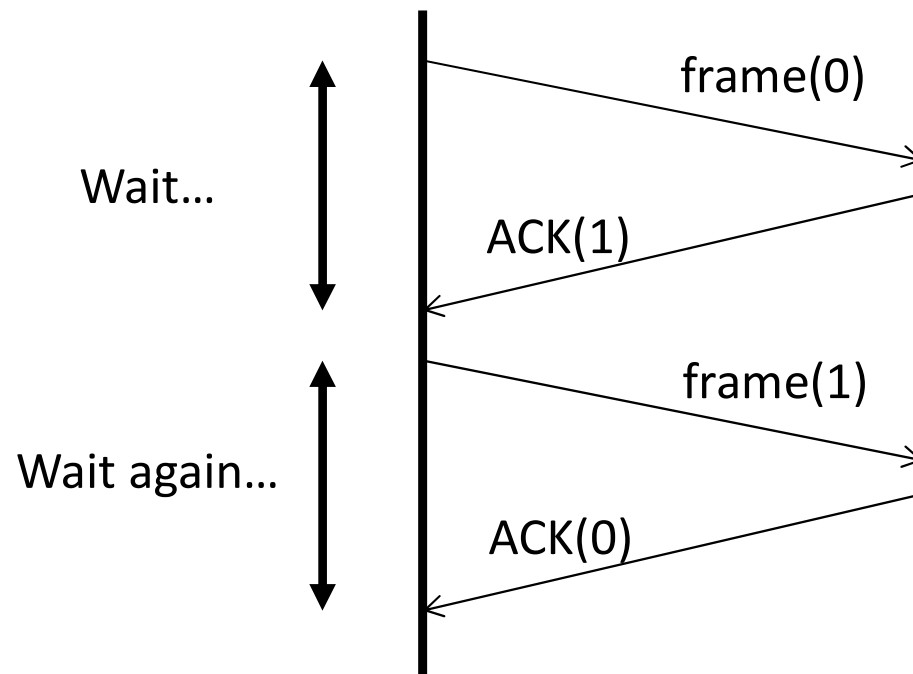


Stop-and-Wait: Performance

- Suppose a system uses stop-and-wait ARQ. The link bandwidth is 1Mbps, and RTT is fixed at 20ms.
- Assuming no error and infinite timeout, what is the throughput? Suppose frame size is 1000 bits.
- What is the ratio of throughput and link bandwidth?
→ bandwidth is wasted!

Stop-and-Wait: Performance

- Problem: When sender sends a frame, it has to wait for ACK before sending the next frame
→ resource waste



Go-Back-N

Stop-and-Wait → Go-Back-N

- It is wasteful to wait for ACK after sending a single frame
- Before waiting an ACK, send multiple frames in order to fill up the link pipe
- If an error occurs at a particular frame, disregard all frames sent after the lost frame, and restart from there → Go-Back-N

Go-Back-N

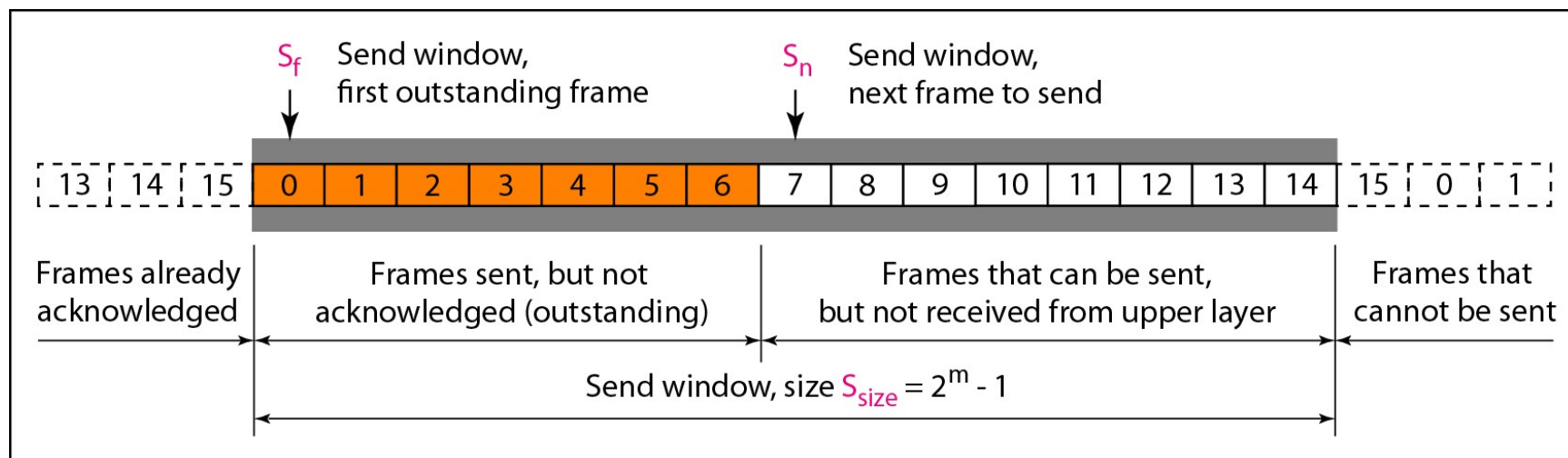
- How many frames should the sender send before receiving an ACK?
- What should be the sequence number range?
- What should we do when an error occurs?

Sliding Window

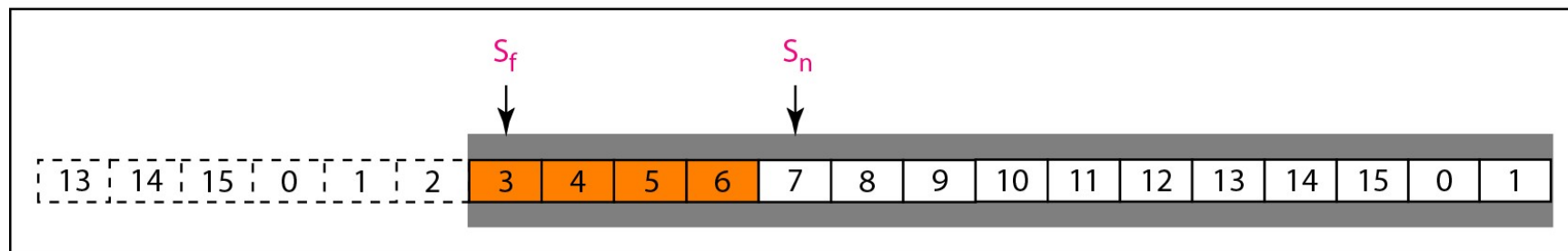
- Manages which frame to send/receive
- Sender and receiver each has a sliding window
 - Sender: send window
 - Receiver: receive window
- Sliding window has a fixed size

Send Window

- Sender can only send frames inside the window
- When ACK is received, the window slides to the right



a. Send window before sliding



b. Send window after sliding

Send Window

- The numbers indicate frame sequence number
- In figure a,
 - Frames 13-15 (left): frame sent, ACK received
 - Frames 0-6: frames sent, ACK not received yet
 - called “outstanding frames”
 - Frames 7-14: frames can be sent, but not ready
 - Frame 15 (right) and after: cannot send yet

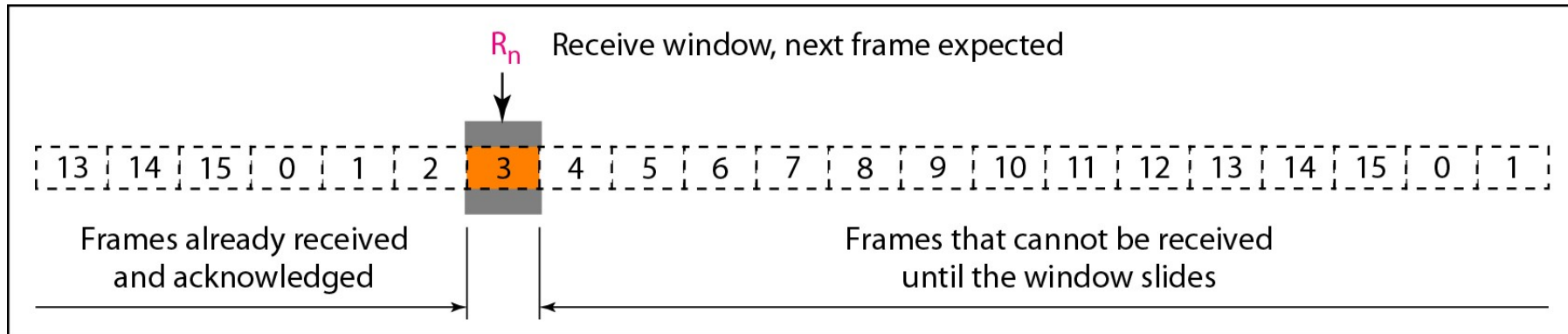
Send Window

- S_f : First outstanding frame
 - The first frame not to have received ACK yet
- S_n : Next frame to read
 - The first frame that is not sent yet
- S_{size} : Send window size

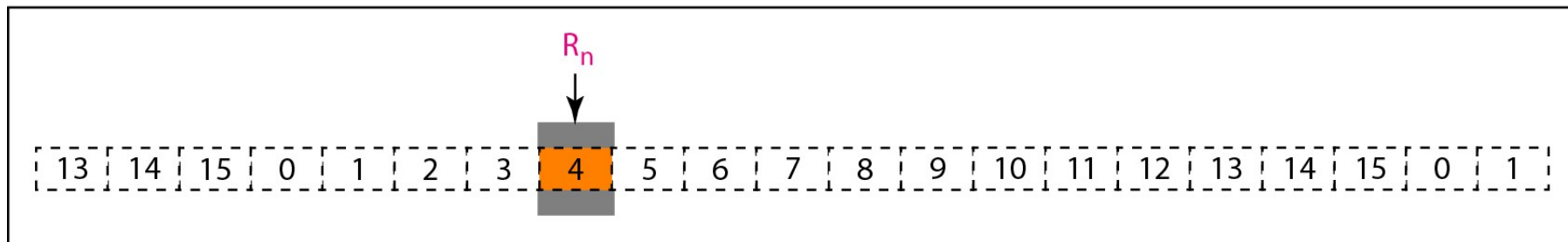
Send Window: Sliding

- When an ACK is received, the window slides to the right
- If ACKs are received for frames 0-2 in figure a, the sliding windows moves as figure b.

Receive Window



a. Receive window



b. Window after sliding

Receive Window

- Receiver can only receive frames inside receive window
- In figure a,
 - frames 13-2: frames received and acknowledged
 - frames 3: frames that can be received
 - frames 4 and after: frames that cannot be received

Sequence Number Range

- What should be the sequence number range of Go-Back-N ARQ?
- If sequence number range is too wide, too many bits are needed to represent sequence number
 - If sequence number range is $[0 \dots N-1]$, then $\log_2 N$ bits are needed
- If sequence number range is too narrow, receiver cannot distinguish new frame from retransmitted frame

Sequence Number Range

- If send window size is N , then the sequence number range must be wider than N !
 - If sequence number uses m bits, the send window size must be less than 2^m
- Example: if send window size is 4, then sequence number range should be at least $[0,4]$

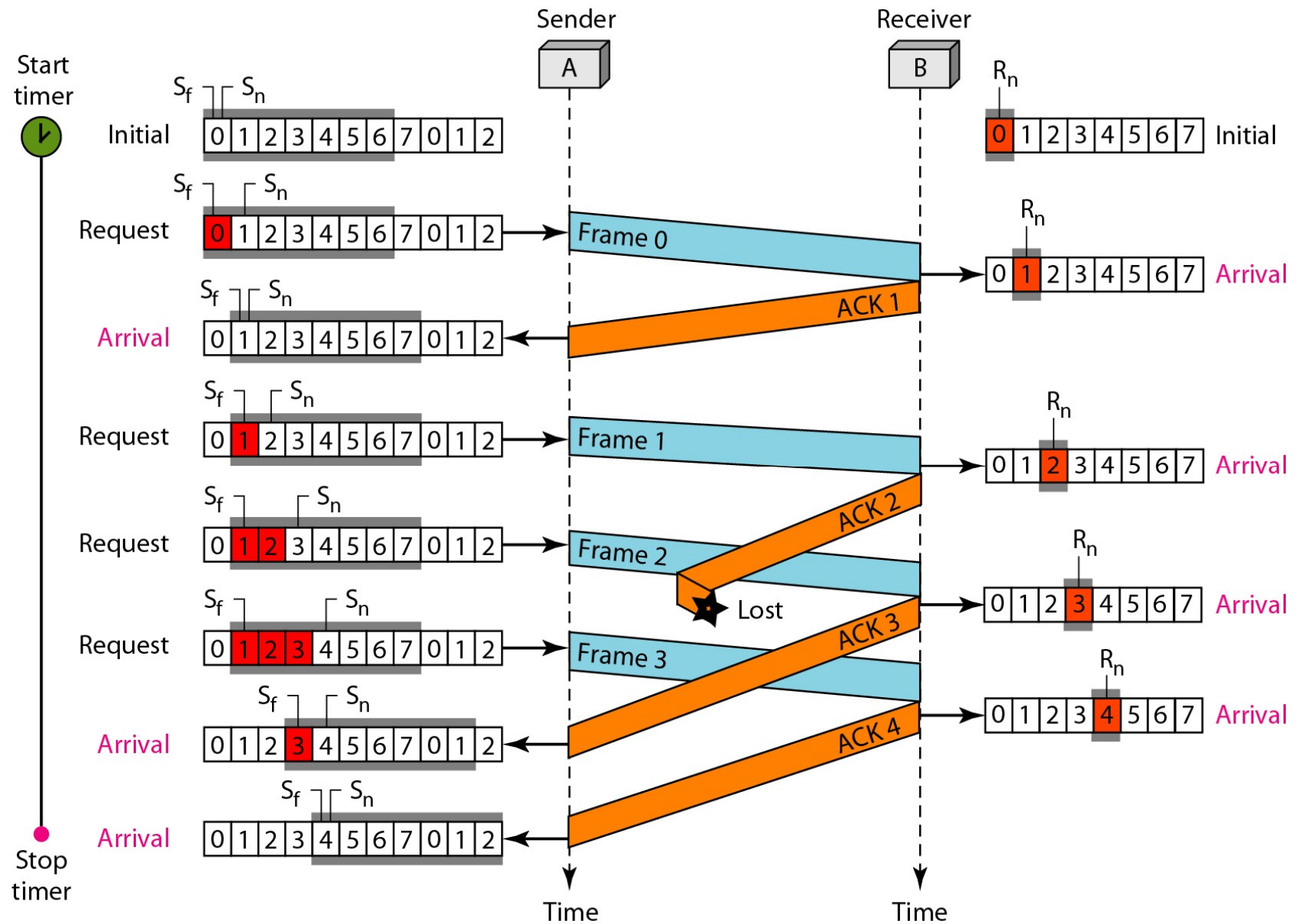
a. Window size $< 2^m$



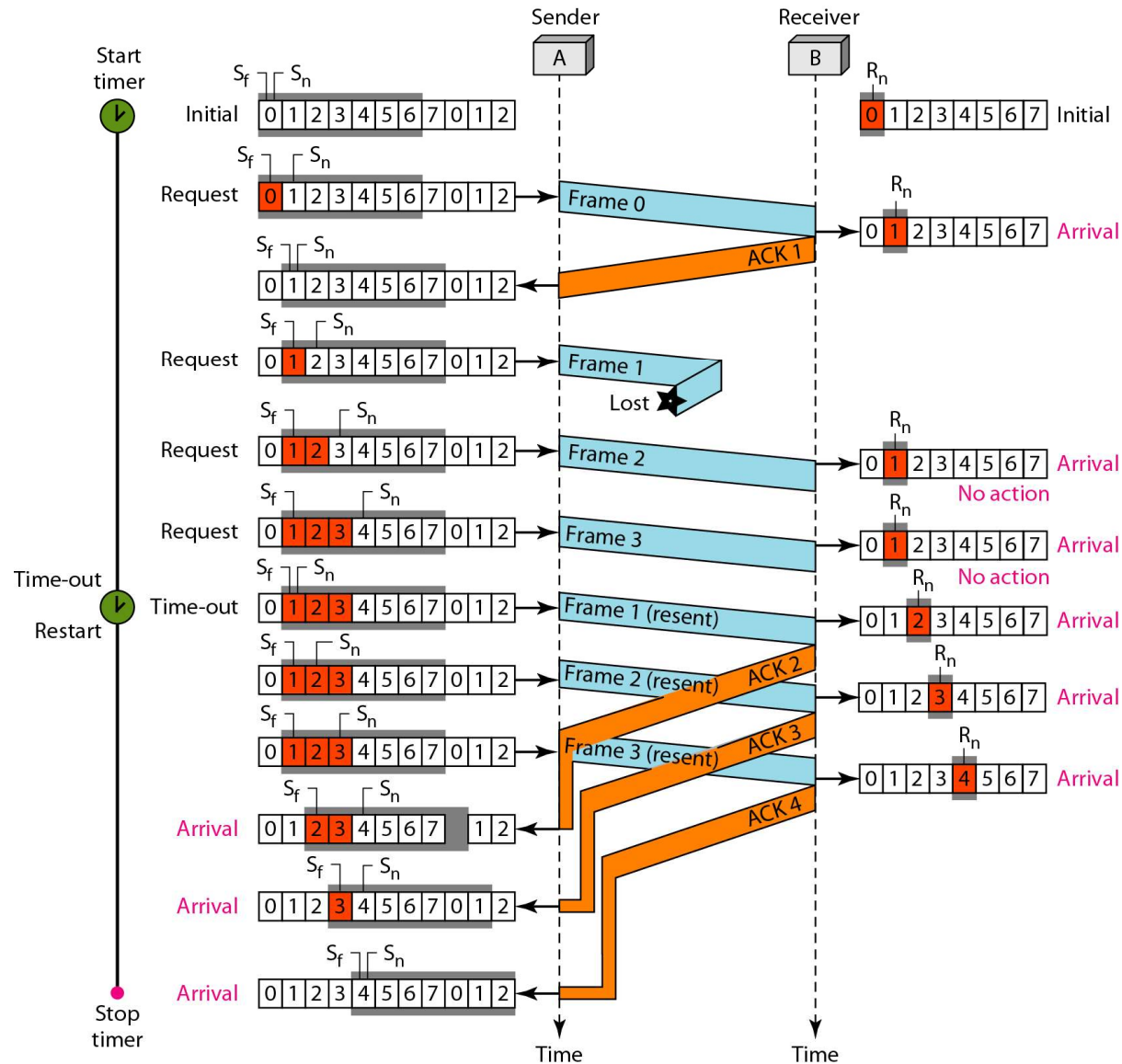
Cumulative ACK

- Go-Back-N uses cumulative ACK
- ACK(n) means "all frames before n have been received"
 - ACK(3) = frames 0, 1, 2 all received successfully
- If ACK(2) is lost but ACK(3) arrives, the sender knows frames 0, 1, 2 are all received
 - No need to retransmit

Go-Back-N: Normal case



Go-Back-N: Timeout case



Go-Back-N: Timer

- Only 1 timer for the oldest outstanding frame S_f
- When ACK is received $\rightarrow$ reset timer for the new S_f
- When all outstanding frames are acknowledged $\rightarrow$ stop timer
- On timeout $\rightarrow$ retransmit all outstanding frames from S_f

Stop-and-Wait vs. Go-Back-N

- Stop-and-Wait is a special case of Go-Back-N
- Go-Back-N with send window size = 1 is Stop-and-Wait

Go-Back-N: Problem

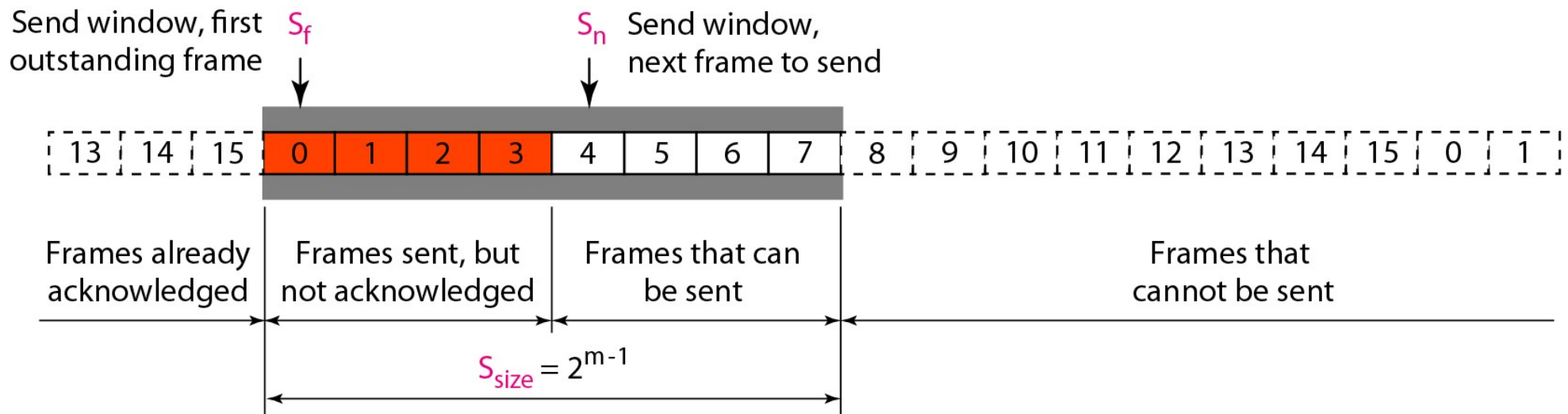
- When timeout occurs, we need to go back and restart from where the error occurred
- This is due to the fact that receive window size = 1
- Example
 - The sender sends frames 1-5
 - Frame 1 is lost, other packets are successfully received
 - Still, go-back-N requires all frames to be retransmitted

Selective Repeat

Go-Back-N → Selective Repeat

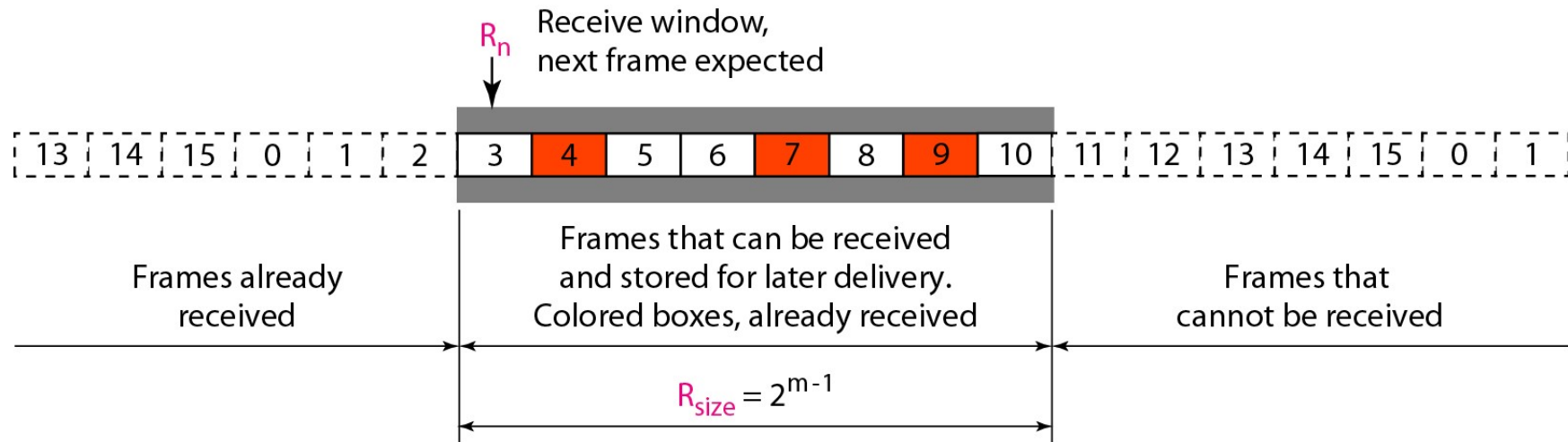
- Overcome problem of Go-Back-N
- Receive window size: N
- Typically, send window size = receive window size

Selective Repeat: send window



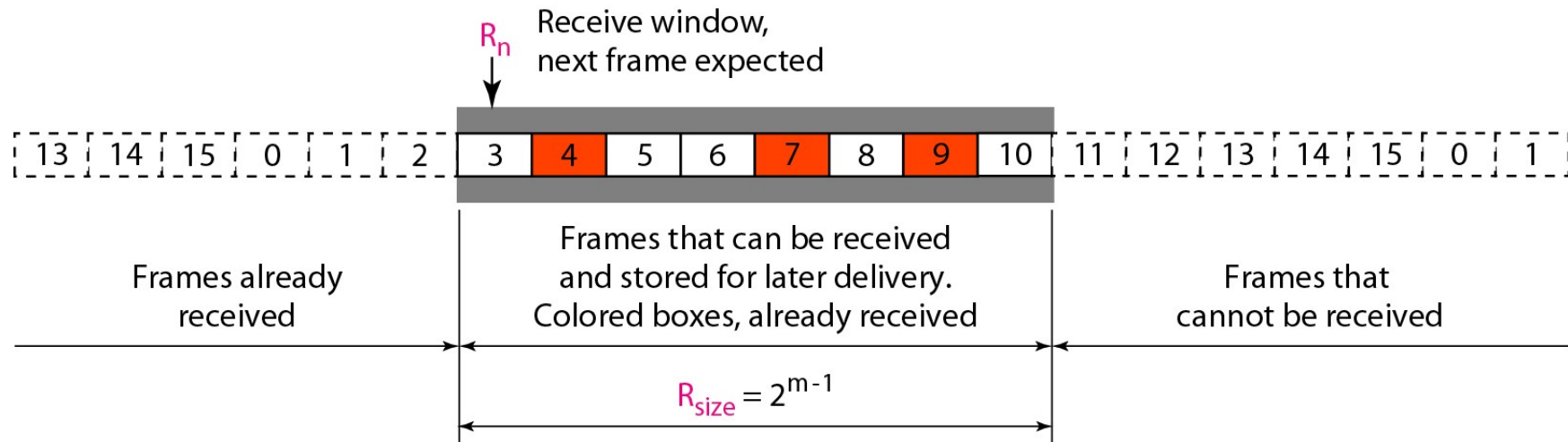
Selective Repeat: receive window

- Even frame 3 is not received, receiver accepts and saves subsequent frames 4, 7, 9
- This is possible because receiver window size is not 1



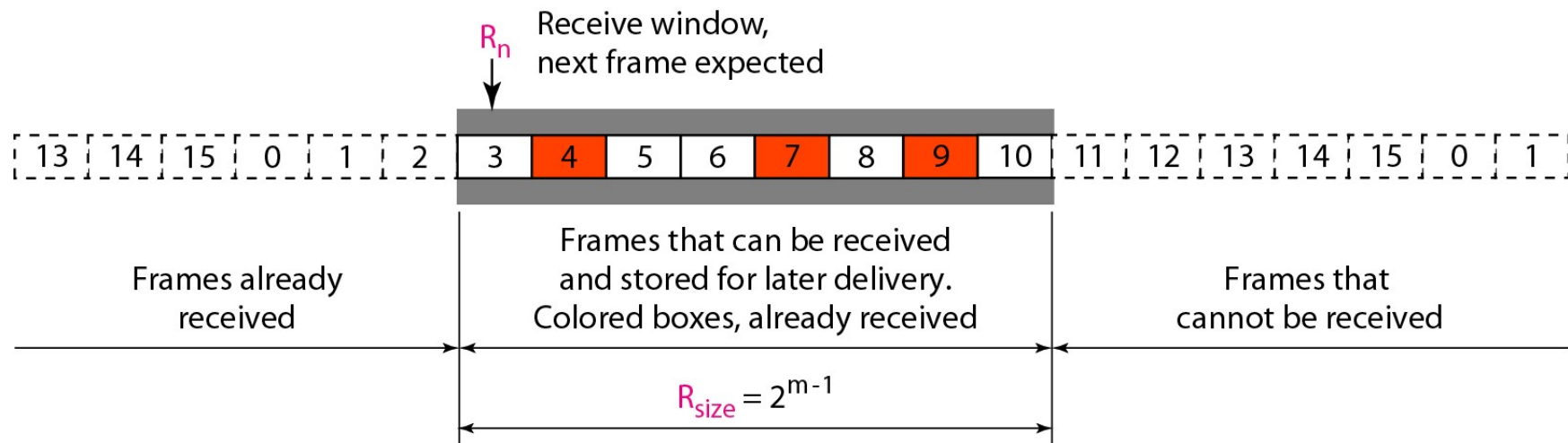
Selective Repeat: receive window

- When frame 3 is received, the receiver sends back ACK(5) to request frame(5), since it already has frame(4)



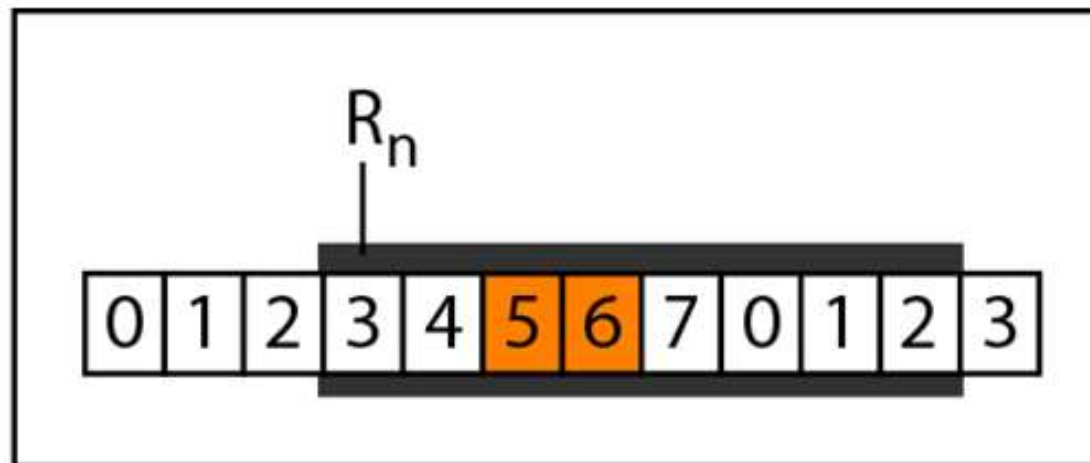
Selective Repeat: sequence numbers

- If sequence number uses m bits, both send window and receive window should be less than or equal to 2^{m-1}



Receiver's action

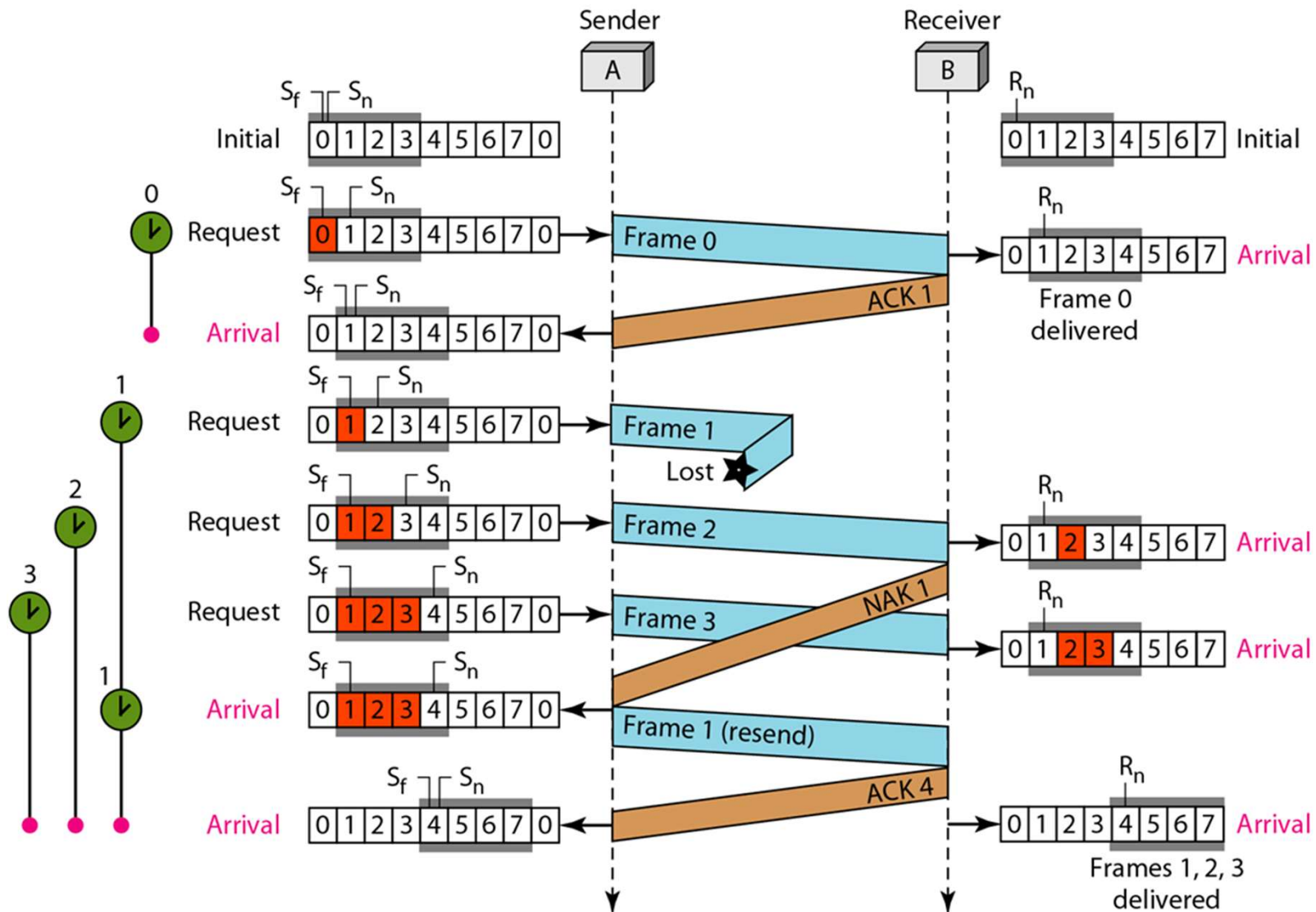
- The receiver window looks like the figure
- Communication sequence
 - frame(3) is received → send ACK(4)
 - frame(4) is received → send ACK(7)
 - since frames 5,6 is already received



Selective Repeat: NAK mechanism

- NAK: Negative acknowledgement
- When the receiver sends NAK(1), it means that a subsequent frame was received without frame(1)
- Without NAK, the sender would have to wait until timeout

Selective Repeat: NAK mechanism



Selective Repeat: NAK mechanism

- When does the receiver send NAK?
 - When a frame is received, but a previous frame is still missing
 - Example: frame 3 is received, but frame 1 has not arrived yet → send NAK(1)
- NAK is sent only once for the same sequence number
 - Receiver does not receive frame(1)
 - Receiver receives frame(2) → send NAK(1)
 - Receiver receives frame(3) → do nothing

Selective Repeat: NAK mechanism

- When does the receiver send ACK?
 - When a frame is received, and receiver has all the previous frames

Selective Repeat: Example

- Receiver's action (frame(0) is the first frame)
 - receives frame(0) → sends ACK(1)
 - receives frame(1) → sends ACK(2)
 - receives frame(3) → sends NAK(2)
 - receives frame(4) → do nothing
 - receives frame(5) → do nothing
 - receives frame(2) → sends ACK(6)

Selective Repeat: Timer

- Each outstanding frame has its own timer (up to N timers)
- On timeout of a specific frame → retransmit only that frame
- More complex to implement than Go-Back-N
 - Go-Back-N: 1 timer
 - Selective Repeat: up to N timers

Additional Topic: Hybrid ARQ

- Hybrid ARQ = ARQ + FEC (Forward Error Correction)
- Traditional ARQ: if error detected → discard frame, request retransmission
- HARQ: if error detected → keep the received frame, request retransmission
 - Combine original and retransmitted frame to improve decoding success
 - Called "soft combining"
- Used in 4G LTE, 5G NR, Wi-Fi 802.11ax
- Much more efficient than traditional ARQ in wireless channels
 - Wireless channels have high error rates
 - Discarding everything on error is wasteful

End of Chapter
